

UNIVERSITÀ DEGLI STUDI DI SALERNO

DIPARTIMENTO DI INFORMATICA

SAfeGuard-AI

*Sistema Intelligente per la Rilevazione e il Clustering
degli Hotspot di Rischio Territoriale*



Autore:

Francesco CARBONE

Mtr. 0512119261

(Team Magma)

Corso:

Fondamenti di I.A.

Anno Accademico 2025/2026



Indice

1	Descrizione dell'Agente Intelligente	2
1.1	Obiettivi del Progetto	2
1.2	Reperibilità e Sviluppo Software	2
1.2.1	Organizzazione del Repository	2
1.3	Specifica PEAS	3
1.3.1	Caratteristiche dell'ambiente	3
2	Raccolta e Preprocessing dei Dati	5
2.1	Origine del Dataset	5
2.2	Adattamento al Contesto Campano	5
2.2.1	Mapping delle Colonne	6
3	Architettura Software	7
3.1	Core Logic: <code>risk_model.py</code>	7
3.1.1	Algoritmo di Calcolo del Risk Score	7
3.2	Data Layer: <code>firestore_repository.py</code>	8
3.3	Interfacce e Versioning	8
3.3.1	Legacy API: <code>routes.py</code> (Branch: <code>dev_render</code>)	8
3.3.2	Demo Interattiva: <code>dashboard.py</code> (Branch: <code>main</code>)	8
3.4	Visualizzazione Geospaziale: Logica di Rendering	9
3.4.1	Stratificazione della Mappa	9
3.4.2	Rappresentazione delle Aree di Impatto (Convex Hull)	9
3.4.3	Interattività e Geofencing	10
4	Sperimentazione e Conclusioni	11
4.1	Analisi Comparativa degli Algoritmi	11
4.1.1	Specifiche dell'Ambiente di Test	12
4.2	Osservazioni sui risultati	12
5	Conclusioni e Sviluppi Futuri	13
5.1	Risultati Conseguiti	13
5.2	Evoluzione del Sistema	13



Capitolo 1

Descrizione dell'Agente Intelligente

1.1 Obiettivi del Progetto

Il progetto **SAfeGuard-AI** mira a sviluppare un agente intelligente capace di trasformare dati grezzi provenienti da segnalazioni di emergenza in informazioni strategiche. L'obiettivo principale è l'identificazione in tempo reale di *Hotspot* di rischio geospaziale per supportare i processi decisionali della protezione civile.

1.2 Reperibilità e Sviluppo Software

Il codice sorgente dell'intero modulo di Intelligenza Artificiale, inclusi i branch di sviluppo e la documentazione tecnica, è ospitato sulla piattaforma GitHub al seguente indirizzo:

<https://github.com/SAfeGuard2025/ModuloAI>

1.2.1 Organizzazione del Repository

Come descritto nei capitoli successivi, il repository è strutturato per riflettere l'evoluzione del progetto:

- **Branch main:** Contiene la versione corrente basata sulla dashboard interattiva Streamlit.
- **Branch dev_render:** Contiene la versione legacy basata su FastAPI per l'integrazione tramite API RESTful.

1.3 Specifica PEAS

Di seguito è riportata la descrizione PEAS dell'ambiente operativo.

Componente	Descrizione nel Progetto
Performance	Misurata tramite metriche di clustering (<i>Silhouette Score</i> , <i>Coesione</i>), accuratezza nella geolocalizzazione e tempo di esecuzione dell'algoritmo.
Environment	Il territorio della regione Campania, caratterizzato da eventi stocastici e dinamici, delimitato da un <i>Bounding Box</i> geografico.
Actuators	Dashboard interattiva (Streamlit), aggiornamento automatico del database Cloud Firestore, generazione di mappe di calore.
Sensors	Input da file CSV storico, streaming di dati in tempo reale dal database Firestore, input manuale dell'operatore.

Tabella 1.1: Analisi PEAS di SAfeGuard-AI

1.3.1 Caratteristiche dell'ambiente

L'ambiente operativo è:

- **Parzialmente Osservabile:** L'agente non ha una visione onnisciente del territorio campano; la sua conoscenza è limitata esclusivamente alle segnalazioni archiviate nel database *Cloud Firestore*.
- **Agente Singolo:** Il compito di analisi, clustering e assegnazione del *Risk Score* è affidato a un unico modulo logico centrale che elabora le informazioni per fornire supporto decisionale.
- **Stocastico:** L'evoluzione delle emergenze non è deterministica. L'arrivo di nuove segnalazioni o il mutamento delle condizioni ambientali presentano una componente di incertezza che l'agente deve gestire attraverso aggiornamenti continui.
- **Episodico:** Ogni entry del dataset viene analizzata come un episodio a sé stante per identificare i cluster attuali, senza che l'azione di raggruppamento presente influenzi la posizione geografica delle segnalazioni future.
- **Dinamico:** L'ambiente è soggetto a variazioni mentre l'agente esegue i propri algoritmi. La natura critica delle emergenze richiede che il sistema elabori i dati in tempo reale per riflettere lo stato più recente della situazione sul campo.



- **Continuo:** Sia le coordinate spaziali (latitudine e longitudine) che la dimensione temporale sono variabili continue. Questa caratteristica giustifica la scelta di algoritmi di clustering basati sulla densità (DBSCAN), capaci di operare in uno spazio non discretizzato.

Capitolo 2

Raccolta e Preprocessing dei Dati

2.1 Origine del Dataset

Il dataset originale utilizzato per l'addestramento iniziale è il "MontcoAlert" disponibile su Kaggle: <https://www.kaggle.com/datasets/mchirico/montcoalert>. Questo dataset contiene chiamate al 911 della contea di Montgomery (PA).

2.2 Adattamento al Contesto Campano

Per simulare un ambiente operativo locale coerente con i requisiti del progetto, i dati originali provenienti dal dataset MontcoAlert sono stati sottoposti a un processo di *Data Augmentation*, *Cleaning* e trasformazione semantica:

- **Rilocazione Spaziale:** Le coordinate originali (Pennsylvania) sono state mappate stocasticamente all'interno del *Bounding Box* della regione Campania (Lat: 39.90-41.55, Lon: 13.85-15.80) per validare le logiche di geofencing dell'applicativo.
- **Randomizzazione delle Categorie:** Le categorie originali sono state sostituite e randomizzate per allinearsi alle tipologie di emergenza gestite dall'applicativo SAfeGuard: "SOS Generico", "Incendio", "Terremoto", "Tsunami", "Alluvione" e "Bomba".
- **Finestra Temporale:** Per testare l'algoritmo di *Recency Score*, i timestamp sono stati rigenerati in modo casuale all'interno dell'ultimo mese rispetto alla data di esecuzione (Dicembre 2025/Gennaio 2026), garantendo la presenza di dati "freschi" per il clustering dinamico.
- **Scrematura e Ristrutturazione Colonne:** Il dataset è stato ottimizzato eliminando le colonne ridondanti dell'originale (`zip`, `e`) e rinominando i campi per una maggiore leggibilità e compatibilità con i modelli Pydantic.



2.2.1 Mapping delle Colonne

Il processo di trasformazione ha portato alla seguente evoluzione della struttura dati:

Colonna Originale	Colonna Attuale	Descrizione
lat	lat	Latitudine rilocata in Campania.
lng	lng	Longitudine rilocata in Campania.
title / desc	Categoria	Mappatura sulle emergenze SAfeGuard.
timeStamp	DataOra	Timestamp randomizzato (ultimo mese).
addr	Indirizzo	Indirizzo generico simulato.
twp	Comune/Circoscrizione	Comune campano associato.
desc	Motivo/Dettaglio	Trasformazione in dettaglio strutturato.

Tabella 2.1: Confronto tra schema sorgente Kaggle e schema SAfeGuard-AI.

Capitolo 3

Architettura Software

Il sistema è modulare e separato in livelli logici per garantire manutenibilità e scalabilità.

3.1 Core Logic: `risk_model.py`

Questo modulo rappresenta il "cervello" dell'agente. Contiene la classe `RiskModel` che implementa la logica di business e gli algoritmi di Machine Learning:

- **Algoritmi:** Implementa sia **DBSCAN** (Density-Based Spatial Clustering of Applications with Noise) che **K-Means** per permettere un confronto tra approcci densità-centrici e centroid-centrici.
- **Calcolo del Rischio:** Definisce una funzione di *Risk Scoring* ponderata basata sulla densità dei cluster e sulla freschezza temporale dei dati, integrata successivamente con la severità intrinseca degli eventi.

3.1.1 Algoritmo di Calcolo del Risk Score

Il calcolo del punteggio di rischio (*Risk Score*) non è una semplice somma di eventi, ma il risultato di una funzione pesata che analizza la criticità di un'area geografica.

All'interno della classe `RiskModel`, il rischio R per un determinato cluster viene calcolato secondo la seguente formula lineare ponderata:

$$R_{score} = (W_{dens} \cdot D_{norm}) + (W_{rec} \cdot T_{norm}) \quad (3.1)$$

Dove:

- W_{dens} è il peso attribuito alla densità spaziale (impostato a **0.6**).
- W_{rec} è il peso attribuito alla freschezza temporale (*recency*, impostato a **0.4**).
- D_{norm} è la densità normalizzata, ovvero il rapporto tra il numero di segnalazioni nel cluster e la soglia di saturazione definita dal modello.
- T_{norm} è il fattore di decadimento temporale (*Time Decay*), che assegna un valore decrescente man mano che l'evento si allontana dal tempo presente.



Questa logica assicura che un cluster molto denso ma obsoleto (es. un evento concluso) abbia un rischio inferiore rispetto a un cluster meno denso ma estremamente recente (es. un'emergenza appena segnalata).

Integrazione della Severità

Mentre il R_{score} definisce la pericolosità di un'area (rischio ambientale), ogni singola segnalazione porta con sé una *Severity* intrinseca S (scala 1-5). Il sistema genera l'indice di priorità finale dell>alert combinando questi due fattori:

$$Priority_{alert} = f(R_{score}, S) \quad (3.2)$$

In questo modo, la Dashboard è in grado di dare massima priorità a segnalazioni ad alta severità che ricadono all'interno di cluster ad alto rischio già identificati dall'agente.

3.2 Data Layer: `firestore_repository.py`

Modulo responsabile della persistenza dei dati. Gestisce l'interazione con il database NoSQL **Google Cloud Firestore**. Utilizza l'SDK `firebase_admin` per operazioni CRUD (Create, Read, Update, Delete) sicure, gestendo sia il caricamento dei report storici che il salvataggio delle zone di rischio identificate.

3.3 Interfacce e Versioning

Il progetto presenta due modalità di interazione distinte, gestite tramite versioning su GitHub.

3.3.1 Legacy API: `routes.py` (Branch: `dev_render`)

Nella versione precedente dell'architettura, il file `routes.py` gestiva l'esposizione di API RESTful tramite il framework **FastAPI**. Questo modulo si occupava di ricevere i dati in formato JSON, validarli tramite schemi **Pydantic** e restituire i risultati dell'analisi all'applicativo mobile **SAfeGuard** (sviluppato in Dart/Flutter).

3.3.2 Demo Interattiva: `dashboard.py` (Branch: `main`)

La versione attuale (Main) sposta il focus sull'analisi interattiva tramite una dashboard web realizzata con **Streamlit**. Il modulo `dashboard.py` funge da controller principale:

- Inizializza il `RiskModel`.
- Visualizza i cluster su mappe interattive **Folium**.
- Permette all'utente di regolare i parametri dell'IA (Epsilon, MinPts, K) in tempo reale.



- Gestisce il ciclo di vita dell'applicazione e il calcolo delle metriche di performance.

3.4 Visualizzazione Geospaziale: Logica di Rendering

La mappatura degli hotspot e dei report è affidata alla libreria **Folium**, integrata in Streamlit tramite il componente `streamlit_folium`. La logica di disegno segue un processo a più livelli per garantire chiarezza e interattività.

3.4.1 Stratificazione della Mappa

Il rendering della mappa in `dashboard.py` avviene sovrapponendo diversi layer informativi:

- **Basemap:** Viene inizializzata una mappa centrata sulle coordinate medie della Campania (Lat: 40.85, Lon: 14.27) con uno zoom predefinito adatto alla visione regionale.
- **Layer dei Report (Markers):** Ogni singola segnalazione presente in Firestore viene rappresentata con un *CircleMarker*. Il colore del marker è dinamico: varia dal blu (bassa severità) al rosso (alta severità), permettendo un'analisi visiva immediata della gravità degli eventi.
- **Layer degli Hotspot (Cluster):** Gli hotspot identificati dal `RiskModel` vengono evidenziati con cerchi di raggio maggiore. Se l'algoritmo utilizzato è DBSCAN, il raggio visualizzato sulla mappa corrisponde esattamente al parametro *Epsilon* utilizzato per il clustering, rendendo l'algoritmo "spiegabile" visivamente.

3.4.2 Rappresentazione delle Aree di Impatto (Convex Hull)

Per definire i confini fisici di un'area a rischio, il sistema utilizza l'algoritmo del **Convex Hull** (Involuppo Convesso) fornito dalla libreria `scipy.spatial`.

- **Logica:** Per ogni cluster con almeno 3 punti, viene calcolato il poligono minimo che contiene tutti i punti del cluster.
- **Rendering:** Il poligono viene disegnato sulla mappa come un'area semitrasparente. Il colore dell'area riflette il *Risk Score* calcolato: aree più scure indicano una combinazione critica di alta densità di eventi e severità elevata.



3.4.3 Interattività e Geofencing

La mappa non è statica; interagisce con la logica di business in due modi:

1. **Tooltips Informativi:** Cliccando su un marker o un'area, l'utente può visualizzare i dettagli del report o le statistiche del cluster (numero di punti, rischio percentuale).
2. **Validazione Spaziale:** Durante l'inserimento di nuove segnalazioni tramite la tab apposita, il sistema verifica se le coordinate cliccate rientrano nel *Campania Bounding Box*. In caso contrario, il marker viene visualizzato sulla mappa con un marker grigio ma verrà ignorato dall'algoritmo per preservare la coerenza del dataset regionale.

Capitolo 4

Sperimentazione e Conclusioni

4.1 Analisi Comparativa degli Algoritmi

Durante la fase di test sulla regione Campania, i due modelli di clustering sono stati messi a confronto per valutarne l'applicabilità nel dominio della gestione emergenze. La tabella 4.1 riassume le principali differenze rilevate.

Parametro	DBSCAN (Density-Based)	K-Means (Centroid-based)
Geometria	Identifica cluster di forma arbitraria.	Crea cluster circolari/sferici.
Rumore	Gestisce i <i>outliers</i> (punti isolati) escludendoli dal rischio.	Forza ogni punto in un cluster, influenzando la media.
Parametri	Richiede <i>Epsilon</i> e <i>MinPts</i> (più intuitivi per il raggio d'azione).	Richiede il numero <i>K</i> di cluster a priori (difficile da stimare nelle emergenze).
Performance	Molto rapido su dataset densi ($\approx 600\text{ms}$).	Più oneroso all'aumentare delle iterazioni ($\approx 4000\text{ms}$).

Tabella 4.1: Confronto tecnico tra algoritmi AI implementati.

Scenario	Algoritmo	Parametri	Cluster	Silhouette	Coesione Media	Tempo (ms)
A: Micro-Hotspot	DBSCAN	$\epsilon = 3.1\text{km}, \text{min} = 12$	23	-0.548	5466 m	1898.17
	K-Means	$K = 20$	20	0.335	13130 m	2299.91
B: Monitoraggio	DBSCAN	$\epsilon = 2.2\text{km}, \text{min} = 8$	73	0.718	1527 m	598.62
	K-Means	$K = 10$	10	0.355	18704 m	1865.03
C: Regionale	DBSCAN	$\epsilon = 2.0\text{km}, \text{min} = 5$	37	0.774	1311 m	453.09
	K-Means	$K = 5$	5	0.365	27240 m	4553.14

Tabella 4.2: Risultati sperimentali e metriche di efficienza.

4.1.1 Specifiche dell'Ambiente di Test

Per garantire la riproducibilità dei risultati cronometrici riportati nella Tabella 4.2, in seguito sono riportate le caratteristiche hardware e software del computer utilizzato:

- **CPU:** [Intel(R) Core(TM) i7-1065G7 CPU @ 1.30GHz (1.50 GHz)]
- **RAM:** [8,00 GB]
- **OS:** [Windows 10]
- **Software Stack:** Python 3.10 con librerie `scikit-learn` 1.5.0 e `pandas` 2.1.3.

Il tempo di esecuzione indicato comprende esclusivamente la fase di *fitting* del modello e il calcolo delle metriche di qualità, escludendo i tempi di latenza di rete del database Firestore.

4.2 Osservazioni sui risultati

Dall'analisi comparativa dei dati raccolti nelle tre simulazioni, emergono diverse considerazioni fondamentali per l'efficacia del sistema **SAfeGuard-AI**:

- **Idoneità del modello:** Il DBSCAN risulta l'algoritmo più idoneo per la gestione delle emergenze. A differenza del K-Means, permette di definire un raggio d'azione reale (espresso in chilometri) e non genera falsi positivi, poiché classifica come *noise* le segnalazioni isolate e distanti dai centri critici.
- **Precisione Geografica e Coesione:** Nello scenario C (Regionale), il DBSCAN ha dimostrato una coesione media di circa **1.3 km**, identificando hotspot estremamente compatti e precisi. Al contrario, il K-Means ha prodotto cluster con una dispersione media di oltre **27 km**, un valore che renderebbe l'informazione del tutto inutile per il dispiegamento tattico dei soccorsi.
- **Sensibilità agli iperparametri:** Lo scenario A evidenzia come il DBSCAN richieda una calibrazione attenta: con un raggio troppo ampio rispetto alla densità dei dati ($\epsilon = 3.1\text{km}$), il *Silhouette Score* scende a valori negativi (-0.548). Questo conferma che il sistema deve essere tarato specificamente sulla densità abitativa dell'area monitorata.
- **Efficienza Computazionale:** È stata rilevata una disparità netta nei tempi di esecuzione, specialmente nello scenario C. Il DBSCAN processa il dataset regionale in appena **453.09 ms**, risultando circa dieci volte più rapido rispetto ai **4553.14 ms** richiesti dal K-Means. Tale reattività è cruciale in contesti *time-critical* per garantire aggiornamenti della dashboard quasi istantanei tramite il database Firestore.
- **Gestione del Carico:** L'osservazione finale per il sistema **SAfeGuard-AI** è che il DBSCAN non solo è più preciso, ma riduce drasticamente la latenza di sistema, permettendo all'agente intelligente di rispondere in tempo reale anche in caso di picchi di segnalazioni durante un'emergenza estesa.

Capitolo 5

Conclusioni e Sviluppi Futuri

Il progetto **SAfeGuard-AI** ha dimostrato come l'integrazione di tecniche di *Unsupervised Learning* e sistemi di monitoraggio in tempo reale possa trasformare radicalmente la gestione delle emergenze territoriali. Superando la logica della semplice visualizzazione puntuale, l'agente intelligente sviluppato è in grado di estrarre conoscenza dai dati, identificando pattern spaziali e livelli di criticità che sfuggirebbero a un'analisi manuale.

5.1 Risultati Conseguiti

L'adozione dell'algoritmo **DBSCAN** si è rivelata vincente per il contesto campano, permettendo di isolare il "rumore" delle segnalazioni sporadiche e focalizzare le risorse della Protezione Civile esclusivamente su hotspot reali e densi. Il sistema di *Risk Scoring* ponderato ha introdotto una dimensione meritocratica nell'intervento: non viene data priorità solo a chi "urla più forte" (maggior numero di segnalazioni), ma a chi si trova in una situazione di pericolo attuale, densa e semanticamente grave.

5.2 Evoluzione del Sistema

Il progetto non è certamente perfetto, ma rappresenta una solida base per implementazioni future di scala maggiore:

- **Integrazione con Flussi Dati Reali:** Attualmente il sistema opera su un dataset sintetico generato per scopi di testing. L'uso di dati reali forniti da enti governativi e non permetterebbe di calibrare i pesi del *Risk Score* su casistiche storiche verificate, migliorando l'accuratezza predittiva del modello.
- **Ottimizzazione dei Percorsi:** L'integrazione con algoritmi di ricerca del cammino minimo potrebbe suggerire alle unità di soccorso le rotte più sicure per raggiungere i cluster identificati, evitando le aree a rischio saturazione.

In conclusione, **SAfeGuard-AI** rappresenta un passo avanti verso una gestione della sicurezza pubblica *data-driven*, dove l'Intelligenza Artificiale non sostituisce l'operatore umano, ma ne potenzia le capacità percettive e decisionali in contesti di alta pressione operativa.



Bibliografia

- [1] Kaggle Dataset. *MontcoAlert - 911 Calls Analysis*. <https://www.kaggle.com/datasets/mchirico/montcoalert>